d5_wdvv_check

November 10, 2026

[2]:
```
"""D5: homogeneous polynomial Frobenius potentials, WDVV, semisimplicity.

Requires SymPy: python -m pip install sympy
Run: python d5_wdvv_check.py

Coordinates t,u,v,w,z have weights 1,3/4,1/2,1/4,5/8.
The Frobenius metric pairs t--w, u--v with coefficient 1,
and z--z with coefficient -4.
"""
import itertools
import sympy as S
```

[3]:
```
s_0, s_1, s_2, s_3, t = S.symbols('s_0 s_1 s_2 s_3 t')
xs=(s_0, s_1,s_2,s_3,t)
p,q,b,c = S.symbols('p q b c')
metric=S.zeros(5)
for i,j in ((0,3),(3,0),(1,2),(2,1)):
    metric[i,j]=1
metric[4,4]=-4
inverse=metric.inv()
unit=S.Rational(1,2)*s_0**2*s_3+s_0*s_1*s_2-2*s_0*t**2
```

[4]:
```
# 16 возможных мономов, независимых от s_0
monoms=(
    s_1**3, s_1**2*s_2*s_3, s_1**2*s_3**3, s_1*s_2**3,
    s_1*s_2**2*s_3**2,s_1*s_2*s_3**4,s_1*s_3**6,s_1*s_3*t**2,
    s_2**4*s_3,s_2**3*s_3**3,s_2**2*s_3**5,s_2**2*t**2,
    s_2*s_3**7,s_2*s_3**2*t**2,s_3**9,s_3**4*t**2,
)
coeff=S.symbols('c0:16')
F_general=unit+sum(a*m for a,m in zip(coeff,monoms))
```

[5]:
```
F_general
```

[5]:
$$c_0 s_1^3 + c_1 s_1^2 s_2 s_3 + c_{10} s_2^2 s_3^5 + c_{11} s_2^2 t^2 + c_{12} s_2 s_3^7 + c_{13} s_2 s_3^2 t^2 + c_{14} s_3^9 + c_{15} s_3^4 t^2 + c_2 s_1^2 s_3^3 + c_3 s_1 s_2^3 + c_4 s_1 s_2^2 s_3^2 + c_5 s_1 s_2 s_3^4 + c_6 s_1 s_3^6 + c_7 s_1 s_3 t^2 + c_8 s_2^4 s_3 + c_9 s_2^3 s_3^3 + \frac{s_0^2 s_3}{2} + s_0 s_1 s_2 - 2 s_0 t^2$$

[6]:
```
def multiplication_matrices(F):
    d3=[[[S.diff(F,xs[i],xs[j],xs[k]) for k in range(5)]
         for j in range(5)] for i in range(5)]
    return [S.Matrix(5,5,lambda r,s:sum(inverse[r,k]*d3[i][s][k]
                                        for k in range(5)))
            for i in range(5)]
```

[7]:
```
def wdvv_constraints(L):
    equations=set()
    for i,j in itertools.combinations(range(1,5),2):
        for expression in L[i]*L[j]-L[j]*L[i]:
            if expression != 0:
                for _,coefficient in S.Poly(expression,s_1,s_2,s_3,t).
 ↪terms():
                    equations.add(S.expand(coefficient))
    return equations
```

[8]:
```
L=multiplication_matrices(F_general)
```

[9]:
```
L
```

[9]:
```
[Matrix([
 [1, 0, 0, 0, 0],
 [0, 1, 0, 0, 0],
 [0, 0, 1, 0, 0],
 [0, 0, 0, 1, 0],
 [0, 0, 0, 0, 1]]),
 Matrix([
 [0, 2*(c1*s_2 + 3*c2*s_3**2), 2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3),
2*(6*c2*s_1*s_3 + c4*s_2**2 + 6*c5*s_2*s_3**2 + 15*c6*s_3**4),    2*c7*t],
 [1,                  2*c1*s_3,                 2*(3*c3*s_2 + c4*s_3**2),
2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3),         0],
 [0,                      6*c0,                                    2*c1*s_3,
2*(c1*s_2 + 3*c2*s_3**2),         0],
 [0,                         0,                                           1,
0,         0],
 [0,                         0,                                           0,
-c7*t/2, -c7*s_3/2]]),
 Matrix([
 [0, 2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3), 2*(5*c10*s_3**4 +␣
 ↪2*c4*s_1*s_3 +
6*c8*s_2**2 + 9*c9*s_2*s_3**2), 2*(20*c10*s_2*s_3**3 + 21*c12*s_3**5 +␣
 ↪c13*t**2
+ 2*c4*s_1*s_2 + 6*c5*s_1*s_3**2 + 9*c9*s_2**2*s_3),
4*c13*s_3*t],
 [0,                 2*(3*c3*s_2 + c4*s_3**2),
6*(c3*s_1 + 4*c8*s_2*s_3 + c9*s_3**3),
```

```
2*(5*c10*s_3**4 + 2*c4*s_1*s_3 + 6*c8*s_2**2 + 9*c9*s_2*s_3**2),
4*c11*t],
 [1,                                  2*c1*s_3,
2*(3*c3*s_2 + c4*s_3**2),
2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3),                                0],
 [0,                                        1,
0,
0,                           0],
 [0,                                        0,
-c11*t,
-c13*s_3*t, -(2*c11*s_2 + c13*s_3**2)/2]]),
 Matrix([
 [0, 2*(6*c2*s_1*s_3 + c4*s_2**2 + 6*c5*s_2*s_3**2 + 15*c6*s_3**4),
2*(20*c10*s_2*s_3**3 + 21*c12*s_3**5 + c13*t**2 + 2*c4*s_1*s_2 +␣
  ↪6*c5*s_1*s_3**2
+ 9*c9*s_2**2*s_3), 6*(10*c10*s_2**2*s_3**2 + 35*c12*s_2*s_3**4 +␣
  ↪84*c14*s_3**6
+ 4*c15*s_3*t**2 + c2*s_1**2 + 4*c5*s_1*s_2*s_3 + 20*c6*s_1*s_3**3 +␣
  ↪c9*s_2**3),
4*t*(c13*s_2 + 6*c15*s_3**2)],
 [0,                         2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3),
2*(5*c10*s_3**4 + 2*c4*s_1*s_3 + 6*c8*s_2**2 + 9*c9*s_2*s_3**2),
2*(20*c10*s_2*s_3**3 + 21*c12*s_3**5 + c13*t**2 + 2*c4*s_1*s_2 +␣
  ↪6*c5*s_1*s_3**2
+ 9*c9*s_2**2*s_3),                                    4*c13*s_3*t],
 [0,                                    2*(c1*s_2 + 3*c2*s_3**2),
2*(c1*s_1 + 2*c4*s_2*s_3 + 2*c5*s_3**3),
2*(6*c2*s_1*s_3 + c4*s_2**2 + 6*c5*s_2*s_3**2 + 15*c6*s_3**4),
2*c7*t],
 [1,                                                                  0,
0,
0,                                           0],
 [0,                                                        -c7*t/2,
-c13*s_3*t,
-t*(c13*s_2 + 6*c15*s_3**2), -(2*c13*s_2*s_3 + 4*c15*s_3**3 + c7*s_1)/
  ↪2]]),
 Matrix([
 [0,    2*c7*t,                 4*c13*s_3*t,               4*t*(c13*s_2 +
6*c15*s_3**2), 2*(2*c13*s_2*s_3 + 4*c15*s_3**3 + c7*s_1)],
 [0,         0,                       4*c11*t,
4*c13*s_3*t,               2*(2*c11*s_2 + c13*s_3**2)],
 [0,         0,                             0,
2*c7*t,                                 2*c7*s_3],
 [0,         0,                             0,
0,                                          -4],
 [1, -c7*s_3/2, -(2*c11*s_2 + c13*s_3**2)/2, -(2*c13*s_2*s_3 +␣
  ↪4*c15*s_3**3 +
```

```
c7*s_1)/2,                                                    0]])]
```

[10]:
```
L=multiplication_matrices(F_general)
assert L[0]==S.eye(5)
equations=wdvv_constraints(L)
print('Coefficient equations from all six WDVV commutators:
  ↪',len(equations))
```

```
Coefficient equations from all six WDVV commutators: 184
```

[11]:
```
equations
```

[11]:
```
{-10*c10 + c13**2/4 + c13*c4 + 3*c7*c9,
 -108*c0*c9 + 12*c1*c4,
 -108*c0*c9 + 4*c1*c4 + 36*c2*c3 + 12*c5,
 -108*c3*c9 + 48*c4*c8,
 -12*c0*c13 - c7**2,
 -12*c1*c3 + 4*c4,
 -12*c13*c7 - 96*c15,
 -144*c0*c8 + 12*c1*c3 + 8*c4,
 -16*c1*c4 + 72*c2*c3,
 -16*c1*c4 + 72*c2*c3 + 24*c5,
 -16*c11*c4 + 24*c13*c3 + 48*c7*c8,
 -160*c10 + 8*c11*c15 + 16*c11*c5 + 2*c13**2 + 8*c13*c4 + 36*c7*c9,
 -2*c1 + c7/2,
 -2*c1*c11 + c11*c7/2 + c13 + 3*c3*c7,
 -2*c1*c11 - c11*c7/2 + 3*c3*c7,
 -2*c11*c15 - 4*c11*c5 + c13**2/2 + 2*c13*c4 + 3*c7*c9,
 -2*c11*c7 - 2*c13,
 -20*c1*c10 - 42*c12 + 36*c2*c9,
 -20*c10*c7 + 120*c11*c6 - 4*c13*c15 - 16*c13*c5,
 -24*c0*c11 + 2*c7,
 -24*c0*c13 - 4*c1*c7 - c7**2,
 -24*c0*c4 + 12*c2,
 -24*c0*c4 + 8*c1**2,
 -24*c1*c8 + 12*c3*c4 + 6*c9,
 -24*c1*c8 + 18*c9,
 -24*c1*c9 - 40*c10 + 144*c2*c8,
 -24*c11*c15 + 4*c13**2 + 4*c13*c4,
 -24*c11*c15 + 48*c11*c5 + 2*c13**2 - 16*c13*c4 - 36*c7*c9,
 -24*c11*c2 - c13*c7,
 -24*c13*c2 - 8*c15*c7 - 8*c5*c7,
 -24*c2 + c7**2,
 -240*c0*c10 - 8*c1*c5 + 48*c2*c4,
 -240*c10 + 48*c11*c5 + 4*c13**2 + 4*c13*c4 + 36*c7*c9,
 -240*c10*c3 + 96*c4*c9 - 192*c5*c8,
 -252*c0*c12 - 60*c1*c6 + 48*c2*c5,
 -252*c12*c3 + 72*c5*c9 - 720*c6*c8,
```

```
-3*c13*c7 - 24*c15,
-36*c0*c3 + 4*c1,
-36*c0*c9 + 4*c1*c4 + 8*c5,
-4*c1**2 + 6*c2,
-4*c1*c13 - 2*c13*c7 - 24*c15,
-4*c1*c7 + c7**2,
-4*c11*c13 + 12*c13*c3,
-4*c11*c13 + 8*c11*c4 - 24*c7*c8,
-4*c11*c4 + 6*c13*c3 + 12*c7*c8,
-4*c13 - 12*c3*c7,
-40*c1*c10 + 72*c2*c9 + 180*c3*c6 - 8*c4*c5,
-40*c10 + 2*c11*c15 + 4*c11*c5 + c13**2/2 + 2*c13*c4 + 9*c7*c9,
-40*c10 + c13**2 + 4*c13*c4 + 12*c7*c9,
-48*c1*c8 + 36*c3*c4,
-6*c2 + c7**2/4,
-60*c0*c10 + 12*c2*c4 + 30*c6,
-60*c1*c10 - 210*c12 + 108*c2*c9 - 180*c3*c6 + 8*c4*c5,
-60*c10 + 12*c11*c5 + c13**2 + c13*c4 + 9*c7*c9,
-60*c10 + 72*c2*c8 - 72*c3*c5 + 12*c4**2,
-72*c0*c5,
-72*c0*c8 + 12*c1*c3 + 2*c4,
-72*c1*c9 + 288*c2*c8 + 144*c3*c5 - 24*c4**2,
-8*c1 + 2*c7,
-8*c1*c11 + 2*c11*c7,
-8*c1*c11 + 2*c11*c7 + 4*c13 + 12*c3*c7,
-8*c1*c11 - 2*c11*c7 + 12*c3*c7,
-8*c1*c11 - 2*c11*c7 - 4*c13,
-8*c1*c13 + 24*c11*c2 - c13*c7 - 24*c15 - 4*c4*c7,
-8*c1*c13 + 48*c11*c2 + 2*c13*c7 - 8*c4*c7,
-8*c1*c13 - 2*c13*c7 - 24*c15 - 4*c4*c7,
-8*c1*c13 - 8*c4*c7,
-8*c1*c4 + 36*c2*c3 - 12*c5,
-8*c1*c5 - 120*c6,
-8*c11*c15 - 16*c11*c5 + 2*c13**2 + 8*c13*c4 + 12*c7*c9,
-8*c11*c7 - 8*c13,
-80*c10*c5 + 84*c12*c4 + 180*c6*c9,
-84*c1*c12 + 60*c10*c2 - 504*c14 - 60*c4*c6 + 16*c5**2,
108*c0*c9 - 12*c1*c4,
108*c0*c9 - 4*c1*c4 - 36*c2*c3 - 12*c5,
108*c3*c9 - 48*c4*c8,
12*c0*c13 + 4*c1*c7 - 24*c2 + c7**2,
12*c0*c13 + c7**2,
12*c1*c3 - 4*c4,
12*c11*c2 + c13*c7 + 2*c4*c7 - 24*c5,
12*c13*c2 + 4*c15*c7 + 8*c5*c7 - 120*c6,
12*c3*c7 - 16*c4,
144*c0*c8 - 12*c1*c3 - 8*c4,
```

```
16*c1*c4 - 72*c2*c3,
16*c1*c4 - 72*c2*c3 - 24*c5,
16*c11*c4 - 24*c13*c3 - 48*c7*c8,
2*c1*c11 + c11*c7/2 + c13,
2*c1*c11 + c11*c7/2 + c13 + 3*c3*c7 - 4*c4,
2*c1*c11 + c11*c7/2 - 4*c4,
2*c1*c11 - 2*c4,
2*c1*c11 - c11*c7/2,
2*c1*c13 + 2*c4*c7,
2*c1*c13 + c13*c7/2 + 6*c15 + c4*c7,
2*c1*c13 - 12*c11*c2 - c13*c7/2 + 2*c4*c7,
2*c1*c13 - 6*c11*c2 + c13*c7/4 + 6*c15 + c4*c7,
2*c11*c4 - 6*c9,
2*c11*c7 + 2*c13,
20*c1*c10 + 42*c12 - 36*c2*c9,
20*c10*c7 + 30*c11*c6 - 210*c12 + 4*c13*c15 + 6*c13*c5,
20*c10*c7 - 120*c11*c6 + 4*c13*c15 + 16*c13*c5,
20*c10*c7 - 168*c12 + 4*c13*c15 + 8*c13*c5,
21*c12*c7 + 15*c13*c6 - 504*c14 + 4*c15**2,
24*c0*c11 - 2*c7,
24*c0*c11 - 8*c1,
24*c0*c13 + 4*c1*c7 + c7**2,
24*c0*c4 - 12*c2,
24*c0*c4 - 8*c1**2,
24*c1*c8 - 12*c3*c4 - 6*c9,
24*c1*c8 - 18*c9,
24*c1*c9 + 40*c10 - 144*c2*c8,
24*c11*c15 - 4*c13**2 - 4*c13*c4,
24*c11*c15 - 48*c11*c5 - 2*c13**2 + 16*c13*c4 + 36*c7*c9,
24*c11*c2 + c13*c7,
24*c13*c2 + 8*c15*c7 + 24*c5*c7 - 480*c6,
24*c13*c2 + 8*c15*c7 + 8*c5*c7,
240*c0*c10 + 8*c1*c5 - 48*c2*c4,
240*c10*c3 - 96*c4*c9 + 192*c5*c8,
252*c0*c12 + 60*c1*c6 - 48*c2*c5,
252*c12*c3 - 72*c5*c9 + 720*c6*c8,
3*c0*c13 + c1*c7 - 6*c2 + c7**2/4,
3*c13*c2 + c15*c7 + 2*c5*c7 - 30*c6,
3*c3*c7 - 4*c4,
36*c0*c3 - 4*c1,
36*c0*c9 - 4*c1*c4 - 8*c5,
4*c1**2 - 6*c2,
4*c1*c13 + 2*c13*c7 + 24*c15,
4*c1*c13 + 24*c11*c2 + 2*c13*c7 + 8*c4*c7 - 48*c5,
4*c1*c13 + c13*c7 + 8*c15 + 4*c4*c7 - 16*c5,
4*c1*c13 + c13*c7 + 8*c4*c7 - 48*c5,
4*c1*c7 - 48*c2 + c7**2,
```

```
4*c1*c7 - c7**2,
4*c11**2 + 24*c11*c3 - 48*c8,
4*c11*c13 + 16*c11*c4 + 24*c7*c8 - 72*c9,
4*c11*c13 + 8*c11*c4 + 12*c13*c3 + 48*c7*c8 - 72*c9,
4*c11*c13 - 12*c13*c3,
4*c11*c13 - 8*c11*c4 + 24*c7*c8,
4*c13 + 12*c3*c7,
40*c1*c10 - 72*c2*c9 - 180*c3*c6 + 8*c4*c5,
48*c1*c8 - 36*c3*c4,
48*c11*c2 + 4*c13*c7 + 8*c4*c7 - 96*c5,
5*c10*c7 - 30*c11*c6 + c13*c15 + 4*c13*c5,
5*c10*c7 - 42*c12 + c13*c15 + 2*c13*c5,
6*c0*c11 - 2*c1,
6*c0*c11 - c7/2,
6*c0*c13 + c1*c7 + c7**2/4,
6*c11*c15 - 12*c11*c5 - c13**2/2 + 4*c13*c4 + 9*c7*c9,
6*c11*c2 + c13*c7/4,
6*c13*c2 + 2*c15*c7 + 2*c5*c7,
6*c13*c2 + 2*c15*c7 + 6*c5*c7 - 120*c6,
60*c0*c10 - 12*c2*c4 - 30*c6,
60*c1*c10 + 210*c12 - 108*c2*c9 + 180*c3*c6 - 8*c4*c5,
60*c10 - 72*c2*c8 + 72*c3*c5 - 12*c4**2,
72*c0*c5,
72*c0*c8 - 12*c1*c3 - 2*c4,
72*c1*c9 - 288*c2*c8 - 144*c3*c5 + 24*c4**2,
8*c1*c11 + 2*c11*c7 + 4*c13,
8*c1*c11 + 2*c11*c7 + 4*c13 + 12*c3*c7 - 16*c4,
8*c1*c11 + 2*c11*c7 - 12*c3*c7,
8*c1*c11 + 2*c11*c7 - 16*c4,
8*c1*c11 - 2*c11*c7,
8*c1*c11 - 2*c11*c7 - 4*c13 - 12*c3*c7,
8*c1*c11 - 8*c4,
8*c1*c13 + 2*c13*c7 + 24*c15 + 4*c4*c7,
8*c1*c13 + 8*c4*c7,
8*c1*c13 - 24*c11*c2 + c13*c7 + 24*c15 + 4*c4*c7,
8*c1*c13 - 48*c11*c2 - 2*c13*c7 + 8*c4*c7,
8*c1*c4 - 36*c2*c3 + 12*c5,
8*c1*c5 + 120*c6,
8*c11*c15 + 16*c11*c5 - 2*c13**2 - 8*c13*c4 - 12*c7*c9,
8*c11*c4 - 24*c9,
80*c10*c5 - 84*c12*c4 - 180*c6*c9,
80*c10*c7 + 120*c11*c6 - 840*c12 + 16*c13*c15 + 24*c13*c5,
84*c1*c12 - 60*c10*c2 + 504*c14 + 60*c4*c6 - 16*c5**2,
84*c12*c7 + 60*c13*c6 - 2016*c14 + 16*c15**2,
c1*c13 + 6*c11*c2 + c13*c7/2 + 2*c4*c7 - 12*c5,
c1*c13 + c13*c7/4 + 2*c15 + c4*c7 - 4*c5,
c1*c13 + c13*c7/4 + 2*c4*c7 - 12*c5,
```

```
 c1*c7 - 12*c2 + c7**2/4,
 c1*c7 - c7**2/4,
 c11**2 + 6*c11*c3 - 12*c8,
 c11*c13 + 2*c11*c4 + 3*c13*c3 + 12*c7*c8 - 18*c9,
 c11*c13 + 4*c11*c4 + 6*c7*c8 - 18*c9,
 c11*c13 - 2*c11*c4 + 6*c7*c8,
 c13 + 3*c3*c7}
```

[14]:
```
# Branch e1^2|0 = 0, i.e. c0 = 0.
branch={a:S.Integer(0) for a in coeff}
branch.update({coeff[3]:b,coeff[11]:c,
               coeff[8]:(c**2+6*b*c)/12})
assert all(S.expand(E.subs(branch))==0 for E in equations)
F_degenerate=S.expand(F_general.subs(branch))
assert S.diff(F_degenerate,s_3,2)==0
print('лист a=0:',F_degenerate)
print('e_3^2 = 0 тождественно, so never semisimple.')
```

```
лист a=0: b*c*s_2**4*s_3/2 + b*s_1*s_2**3 + c**2*s_2**4*s_3/12 +␣
 ↪c*s_2**2*t**2 +
s_0**2*s_3/2 + s_0*s_1*s_2 - 2*s_0*t**2
e_3^2 = 0 тождественно, so never semisimple.
```

[15]:
```
F_degenerate
```

[15]:
$$\frac{bcs_2^4s_3}{2} + bs_1s_2^3 + \frac{c^2s_2^4s_3}{12} + cs_2^2t^2 + \frac{s_0^2s_3}{2} + s_0s_1s_2 - 2s_0t^2$$

[8]:
```
# The second WDVV branch, c0=p, c3=q; a=6p.
generic={a:S.Integer(0) for a in coeff}
generic.update({
    coeff[0]:p,
    coeff[1]:9*p*q,
    coeff[2]:54*p**2*q**2,
    coeff[3]:q,
    coeff[4]:27*p*q**2,
    coeff[7]:36*p*q,
    coeff[8]:S.Rational(9,4)*q**2,
    coeff[9]:27*p*q**3,
    coeff[10]:S.Rational(1458,5)*p**2*q**4,
    coeff[11]:3*q,
    coeff[13]:-108*p*q**2,
    coeff[14]:S.Rational(13122,7)*p**4*q**6,
    coeff[15]:486*p**2*q**3,
})
assert all(S.expand(E.subs(generic))==0 for E in equations)
```

[9]:
```
# Trace pairing B_ij = Tr(L_i L_j) at (t,u,v,w,z)=(0,0,0,1,0).
point={t:0,u:0,v:0,w:1,z:0}
Lg=[S.Matrix(M.subs(generic).subs(point)) for M in L]
trace_pairing=S.Matrix(5,5,lambda i,j:S.trace(Lg[i]*Lg[j]))
delta=S.factor(trace_pairing.det())
print('Trace determinant at (0,0,0,1,0):',delta)
assert S.simplify(delta-2**25*3**25*p**10*q**15)==0
print('With a=6p, this is 6^15 * a^10 * q^15.')
print('CONCLUSION: generic semisimplicity is equivalent to p*q != 0.')
```

```
Trace determinant at (0,0,0,1,0): 284302880299297013 76*p**10*q**15
With a=6p, this is 6^15 * a^10 * q^15.
CONCLUSION: generic semisimplicity is equivalent to p*q != 0.
```

[ ]: